MAST32001 Computational Statistics

Exercise 6 – Gibbs sampling, HMC, Stan, and variational inference

Zhi Zhou

2026-10-13

Exercise 1: Gibbs sampling with a latent variable

Suppose

$$Y_i = \theta + Z_i + \varepsilon_i, \qquad i = 1, ..., n,$$

where

$$Z_i \overset{\text{iid}}{\sim} N(0, \tau^2), \qquad \varepsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2),$$

and

$$\theta \sim N(0, s_0^2)$$

Assume that τ^2, σ^2, s_0^2 are known.

```
library(ggplot2)
```

```
Warning: package 'ggplot2' was built under R version 4.6.1
```

```
library(patchwork)
```

```
Warning: package 'patchwork' was built under R version 4.6.1
```

```
knitr::opts_chunk$set(
  echo = TRUE,
  message = FALSE,
  warning = FALSE
)
```

Task 1

Derive the full conditional distribution of each Z_i given θ and y_i.

Answer:

By Bayes' theorem, the full conditional distribution of Z_i is proportional to the likelihood times the prior:

$$p(Z_i \mid \theta, y_i) \propto p(y_i \mid Z_i, \theta) p(Z_i)$$

Give Z_i and θ, $Y_i \sim N(\theta + Z_i, \sigma^2)$. Also, $Z_i \sim N(0, \tau^2)$. Ignoring constant terms, we can write the jointly density:

$$p(Z_i \mid \theta, y_i) \propto exp\left(-\frac{(y_i - \theta - Z_i)^2}{2\sigma^2}\right) exp\left(-\frac{Z_i^2}{2\tau^2}\right)$$

Expanding the exponent and completing the square with respect to Z_i:

$$-\frac{1}{2}\left[\frac{Z_i^2 - 2Z_i(y_i - \theta) + (y_i - \theta)^2}{\sigma^2} + \frac{Z_i^2}{\tau^2}\right] = -\frac{1}{2}\left[Z_i^2(\frac{1}{\sigma^2} + \frac{1}{\tau^2}) - 2Z_i\frac{y_i - \theta}{\sigma^2} + C\right]$$

This is the kernel of a normal distribution with:

- Variance: $\Sigma_{Z_i} = \frac{\sigma^2\tau^2}{\sigma^2+\tau^2}$.
- Mean: $\mu_{Z_i} = \frac{\tau^2}{\sigma^2+\tau^2}(y_i - \theta)$.

Thus,

$$\boxed{Z_i \mid \theta, y_i \sim N\left(\frac{\tau^2}{\sigma^2 + \tau^2}(y_i - \theta), \frac{\sigma^2\tau^2}{\sigma^2 + \tau^2}\right).}$$

Task 2

Derive the full conditional distribution of θ given $Z_1, ..., Z_n$ and the data.

Answer:

Given the latent variables,

$$Y_i \mid \theta, Z_i \sim N(\theta + Z_i, \sigma^2),$$

and the prior is

$$\theta \sim N(0, s_0^2).$$

Therefore,

$$p(\theta \mid Z, y) \propto \exp\left\{-\frac{\theta^2}{2s_0^2} - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \theta - Z_i)^2\right\}.$$

Hence,

$$\theta \mid Z, y \sim N\left(\frac{s_0^2}{\sigma^2 + n s_0^2} \sum_{i=1}^{n} (y_i - Z_i), \frac{\sigma^2 s_0^2}{\sigma^2 + n s_0^2}\right).$$

Task 3

Implement a Gibbs sampler in R.

Answer:

At iteration t:

1. Given $\theta^{(t-1)}$, draw

$$Z_i^{(t)} \sim N\left(\frac{\tau^2}{\sigma^2 + \tau^2}(y_i - \theta^{(t-1)}), \frac{\sigma^2 \tau^2}{\sigma^2 + \tau^2}\right).$$

2. Given $Z_1^{(t)}, \ldots, Z_n^{(t)}$, draw

$$\theta^{(t)} \sim N\left(\frac{s_0^2}{\sigma^2 + n s_0^2} \sum_{i=1}^{n} (y_i - Z_i^{(t)}), \frac{\sigma^2 s_0^2}{\sigma^2 + n s_0^2}\right).$$

The R implementation is:

```r
gibbs_latent <- function(y,
                         tau,
                         sigma,
                         s0,
                         n_iter = 100000,
                         burn_in = 10000,
                         theta_init = 0) {
  n <- length(y)
  tau2 <- tau^2
  sig2 <- sigma^2
  s02 <- s0^2

  var_z <- 1 / (1 / tau2 + 1 / sig2)
  mean_z_factor <- tau2 / (tau2 + sig2)

  var_theta <- 1 / (1 / s02 + n / sig2)
  mean_theta_factor <- s02 / (sig2 + n * s02)

  theta <- theta_init
  theta_draws <- numeric(n_iter)

  for (b in seq_len(n_iter)) {
```

```
    # Draw latent variables conditional on theta and the data
    z <- rnorm(n, mean = mean_z_factor * (y - theta), sd = sqrt(var_z))

    # Draw theta conditional on the latent variables and the data
    theta <- rnorm(1, mean = mean_theta_factor * sum(y - z), sd = sqrt(var_theta))

    theta_draws[b] <- theta
  }

  keep <- seq(burn_in + 1, n_iter)
  list(theta = theta_draws[keep], all_theta = theta_draws)
}
```

Task 4

Simulate a data set with $n = 50, \theta = 2, \tau = 1, \sigma = 0.5$, and $s_0 = 5$. Run the sampler and estimate the posterior mean and standard deviation of θ.

Answer:

```
set.seed(10110104)

n <- 50
theta_true <- 2
tau <- 1
sigma <- 0.5
s0 <- 5

z <- rnorm(n, mean = 0, sd = tau)
eps <- rnorm(n, mean = 0, sd = sigma)
y <- theta_true + z + eps

post <- gibbs_latent(
  y = y,
  tau = tau,
  sigma = sigma,
  s0 = s0,
  n_iter = 100000,
  burn_in = 10000
)
```

The observed sample mean is 2.0713. The retained posterior sample contains 90,000 draws.

```r
gibbs_summary <- data.frame(
  `Posterior mean` = round(mean(post$theta), 4),
  `Posterior SD` = round(sd(post$theta), 4),
  `95% lower` = round(as.numeric(quantile(post$theta, 0.025)), 4),
  `95% upper` = round(as.numeric(quantile(post$theta, 0.975)), 4),
  check.names = FALSE
)
gibbs_summary
```

Table 1 | Task 4: Posterior estimates for theta from the Gibbs sampler.

Posterior mean	Posterior SD	95% lower	95% upper
2.0683	0.1572	1.7593	2.3731

Thus, the estimated posterior mean is 2.0683, and the estimated posterior standard deviation is 0.1572.

```r
nature_theme <- theme_classic(base_size = 8, base_family = "sans") +
  theme(
    axis.text = element_text(size = 7, color = "black"),
    axis.title = element_text(size = 8),
    axis.ticks = element_line(linewidth = 0.2, color = "black"),
    axis.line = element_line(linewidth = 0.2, color = "black"),
    plot.margin = margin(t = 3, r = 5, b = 3, l = 3, unit = "pt")
  )

trace_df <- data.frame(
  iteration = seq_along(post$all_theta),
  theta = post$all_theta
)

p_trace <- ggplot(trace_df, aes(x = iteration, y = theta)) +
  geom_line(color = "steelblue", linewidth = 0.1) +
  geom_hline(
    yintercept = theta_true,
    color = "tomato",
    linetype = "dashed",
    linewidth = 0.35
  ) +
  scale_x_continuous(expand = expansion(mult = c(0.02, 0.02))) +
  labs(x = "Iteration", y = expression(theta)) +
  nature_theme

post_df <- data.frame(theta = post$theta)
```

```r
p_posterior <- ggplot(post_df, aes(x = theta)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 80,
    fill = "grey88",
    color = "white",
    linewidth = 0.1
  ) +
  geom_vline(
    xintercept = theta_true,
    color = "tomato",
    linetype = "dashed",
    linewidth = 0.35
  ) +
  labs(x = expression(theta), y = "Density") +
  nature_theme

diagnostic_plot <- p_trace + p_posterior +
  plot_annotation(tag_levels = "a") &
  theme(plot.tag = element_text(face = "bold", size = 9, family = "sans"))

diagnostic_plot
```

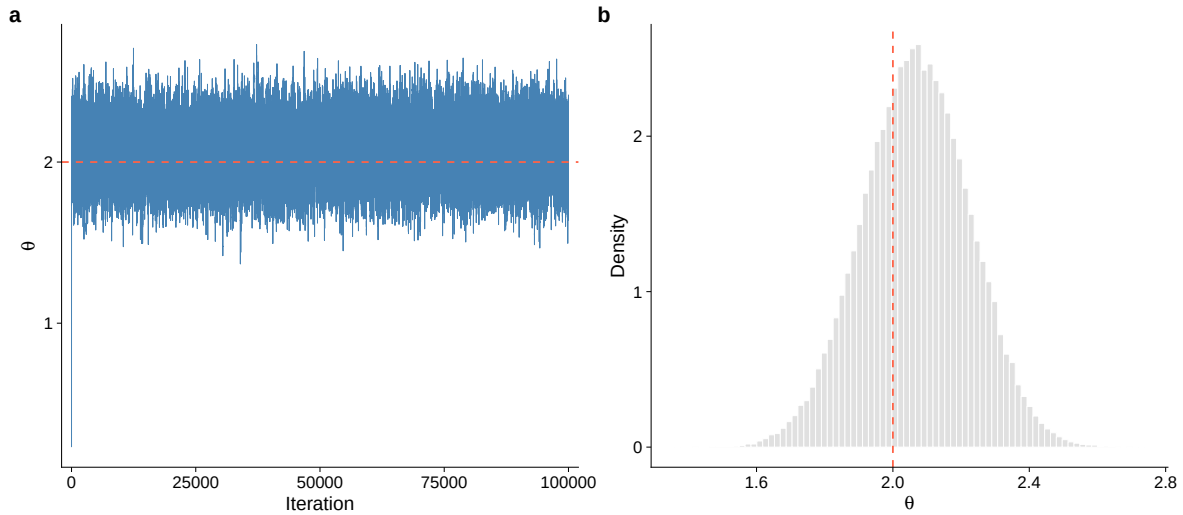


Figure 1: Task 4: Gibbs sampler diagnostics. a, Trace of theta draws. b, Posterior histogram for theta; the red dashed line denotes the true value theta = 2.

Task 5

The latent variables are not actually needed for this model. Integrate out Z_i and identify the marginal sampling distribution of $Y_i|\theta$. Explain one advantage and one disadvantage of

introducing the Z_i's anyway.

Answer:

Because Z_i and ε_i are independent normal variables,

$$Z_i + \varepsilon_i \sim N(0, \tau^2 + \sigma^2).$$

Therefore,

$$Y_i \mid \theta \overset{iid}{\sim} N(\theta, \tau^2 + \sigma^2).$$

Combining this marginal likelihood with the prior $\theta \sim N(0, s_0^2)$ gives

$$\theta \mid y \sim N(\mu_n, \nu_n),$$

where

$$\mu_n = \frac{ns_0^2}{\tau^2 + \sigma^2 + ns_0^2}\bar{y},$$

and

$$\nu_n = \frac{s_0^2(\tau^2 + \sigma^2)}{\tau^2 + \sigma^2 + ns_0^2}.$$

```
marginal_var <- tau^2 + sigma^2
posterior_precision <- 1 / s0^2 + n / marginal_var
post_var_exact <- 1 / posterior_precision
post_mean_exact <- post_var_exact * (n * mean(y) / marginal_var)
post_sd_exact <- sqrt(post_var_exact)
```

```
comparison <- data.frame(
  Method = c("Gibbs sampler", "Exact marginal posterior"),
  `Posterior mean` = round(c(mean(post$theta), post_mean_exact), 4),
  `Posterior SD` = round(c(sd(post$theta), post_sd_exact), 4),
  check.names = FALSE
)
comparison
```

Table 2 | Task 5: Comparison between Gibbs estimates and the exact marginal posterior.

Method	Posterior mean	Posterior SD
Gibbs sampler	2.0683	0.1572
Exact marginal posterior	2.0692	0.1580

Advantage of introducing Z_i: data augmentation produces simple Gaussian full conditionals and makes the Gibbs sampler easy to implement. The latent variables also have a natural random-effect interpretation.

Disadvantage of introducing Z_i: the auxiliary Z_i's are unnecessary when inference concerns only θ. They add computation, storage, Monte Carlo error, and potentially slower MCMC mixing, whereas integrating them out gives the exact posterior directly.

Exercise 2: HMC and the geometry of a correlated target

Consider the bivariate normal target

$$\theta = \begin{pmatrix} \theta_1 \\ \theta_2 \end{pmatrix} \sim N_2 \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma \right), \qquad \Sigma = \begin{pmatrix} 1 & 0.98 \\ 0.98 & 1 \end{pmatrix}.$$

Ignoring additive constants,

$$\log \pi(\theta) = -\frac{1}{2} \theta^\top \Sigma^{-1} \theta.$$

Task 1

Derive $\nabla_\theta \log \pi(\theta)$.

Answer:

Ignoring additive constants, the log-density is

$$\log \pi(\theta) = -\frac{1}{2} \theta^\top \Sigma^{-1} \theta.$$

Differentiating with respect to θ gives

$$\nabla_\theta \log \pi(\theta) = -\Sigma^{-1} \theta.$$

Since

$$\Sigma^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix},$$

the gradient is

$$\nabla_\theta \log \pi(\theta) = -\frac{1}{1-\rho^2} \begin{pmatrix} \theta_1 - \rho\theta_2 \\ \theta_2 - \rho\theta_1 \end{pmatrix}.$$

For $\rho = 0.98$,

$1 - \rho^2 = 0.0396$.

Task 2

Plot contours of the target. Explain why independent random-walk proposals in the two coordinate directions are poorly matched to its geometry.

Answer:

The eigenvectors of Σ are

$$q_u = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad q_v = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix},$$

with eigenvalues

$$\lambda_u = 1 + \rho = 1.98, \qquad \lambda_v = 1 - \rho = 0.02.$$

Thus, the contours are ellipses strongly elongated along the diagonal direction $\theta_1 = \theta_2$.

```
rho <- 0.98
Sigma_ex2 <- matrix(c(1, rho, rho, 1), nrow = 2)
prec_ex2 <- solve(Sigma_ex2)

grid_vals <- seq(-2.6, 2.6, length.out = 350)
contour_df <- expand.grid(theta1 = grid_vals, theta2 = grid_vals)
theta_mat <- as.matrix(contour_df)
quad_mat <- theta_mat %*% prec_ex2
contour_df$density <- exp(-0.5 * rowSums(quad_mat * theta_mat)) /
  (2 * pi * sqrt(det(Sigma_ex2)))

radii2 <- c(0.25, 0.5, 1, 2, 4, 6, 9)
contour_levels <- exp(-radii2 / 2) /
  (2 * pi * sqrt(det(Sigma_ex2)))

long_axis <- data.frame(theta1 = c(-2.3, 2.3), theta2 = c(-2.3, 2.3))
short_axis <- data.frame(theta1 = c(-0.22, 0.22),
                         theta2 = c(0.22, -0.22))
proposal_df <- data.frame(t = seq(0, 2 * pi, length.out = 300))
proposal_df$theta1 <- 0.15 * cos(proposal_df$t)
proposal_df$theta2 <- 0.15 * sin(proposal_df$t)
```

```
theme_ex2 <- theme_classic(base_size = 8, base_family = "sans") +
  theme(
    axis.text = element_text(size = 7, color = "black"),
    axis.title = element_text(size = 8),
    axis.ticks = element_line(linewidth = 0.2, color = "black"),
    axis.line = element_line(linewidth = 0.2, color = "black"),
    plot.margin = margin(
      t = 3,
      r = 5,
      b = 3,
      l = 3,
      unit = "pt"
    )
  )

p_geometry <- ggplot(contour_df, aes(x = theta1, y = theta2)) +
  geom_contour(
    aes(z = density),
    breaks = contour_levels,
    color = "#0072B2",
    linewidth = 0.25
  ) +
  geom_line(data = long_axis,
            color = "#D55E00",
            linewidth = 0.4) +
  geom_line(data = short_axis,
            color = "#C82423",
            linewidth = 0.4) +
  annotate(
    "text",
    x = 2.35,
    y = 2.35,
    label = "u",
    hjust = 0,
    size = 3,
    color = "#D55E00"
  ) +
  annotate(
    "text",
    x = 0.25,
    y = -0.25,
    label = "v",
    hjust = 0,
    size = 3,
```

```
    color = "#C82423"
  ) +
  coord_equal(
    xlim = c(-2.6, 2.6),
    ylim = c(-2.6, 2.6),
    expand = FALSE
  ) +
  labs(x = expression(theta[1]), y = expression(theta[2])) +
  theme_ex2

p_rw <- ggplot(contour_df, aes(x = theta1, y = theta2)) +
  geom_contour(
    aes(z = density),
    breaks = contour_levels,
    color = "grey70",
    linewidth = 0.25
  ) +
  geom_path(
    data = proposal_df,
    color = "black",
    linetype = "dashed",
    linewidth = 0.4
  ) +
  annotate(
    "segment",
    x = 0,
    y = 0,
    xend = 0.18,
    yend = 0,
    arrow = arrow(length = unit(0.08, "cm")),
    linewidth = 0.3
  ) +
  annotate(
    "segment",
    x = 0,
    y = 0,
    xend = 0,
    yend = 0.18,
    arrow = arrow(length = unit(0.08, "cm")),
    linewidth = 0.3
  ) +
  annotate(
    "text",
    x = 0.25,
```

```
    y = -0.12,
    label = "axis-aligned RW",
    hjust = 0,
    size = 2.5
  ) +
  coord_equal(
    xlim = c(-2.6, 2.6),
    ylim = c(-2.6, 2.6),
    expand = FALSE
  ) +
  labs(x = expression(theta[1]), y = expression(theta[2])) +
  theme_ex2

contour_figure <- p_geometry + p_rw +
  plot_annotation(tag_levels = "a") &
  theme(plot.tag = element_text(
    face = "bold",
    size = 9,
    family = "sans"
  ))

contour_figure
```

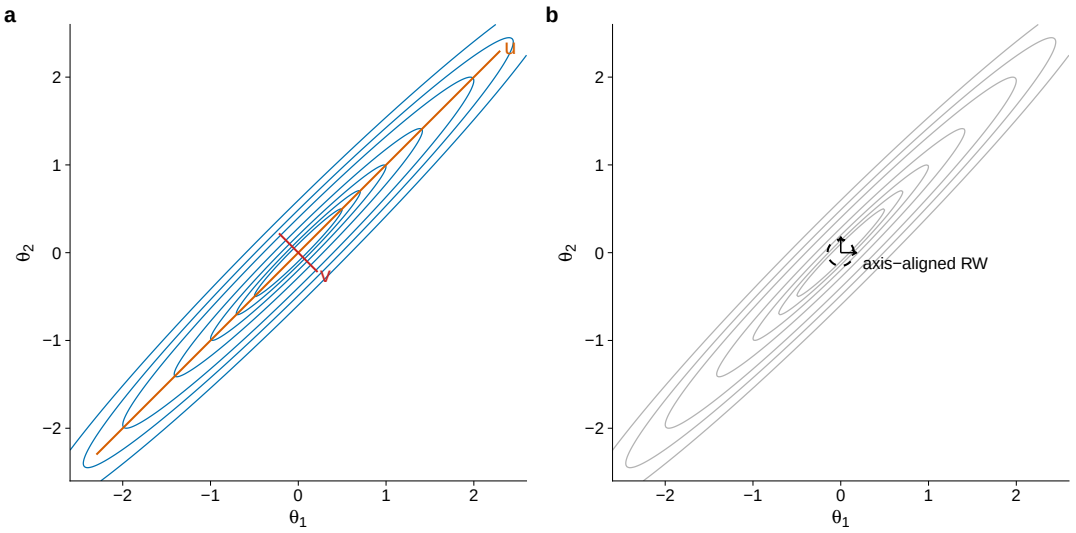


Figure 2: Task 2: Geometry of the correlated target. a, Highly elongated Gaussian contours with principal directions u and v. b, The same contours with a small axis-aligned random-walk proposal scale; the dashed circle is constrained by the narrow v direction but is inefficient for movement along the u direction.

The kind of proposal doesn't match the target well, because:

The proposal covariance is diagonal, but the target has a strong correlation between the two coordinates. They don't move independently. Instead, they tend to move together.

When they move horizontally or vertically, it looks like only one coordinate changes. In fact, the position along both the long axis and the short axis shifts.

Also, to avoid moving too far in the narrow direction, the step size has to be very small. So exploring along the long diagonal becomes very slow.

A better moves is to let both coordinates move together in a certain ratio. Independent coordinate proposal can't easily make that kind of move.

Task 3

Write a function in R that returns the log target and another that returns its gradient.

Answer:

```r
make_bvn_target <- function(rho = 0.98) {
  Sigma <- matrix(c(1, rho, rho, 1), nrow = 2)
  precision <- solve(Sigma)

  log_pi <- function(theta) {
    theta <- as.vector(theta)
    - 0.5 * as.numeric(t(theta) %*% precision %*% theta)
  }

  grad_log_pi <- function(theta) {
    theta <- as.vector(theta)
    as.vector(-precision %*% theta)
  }

  list(
    Sigma = Sigma,
    precision = precision,
    log_pi = log_pi,
    grad_log_pi = grad_log_pi
  )
}

target <- make_bvn_target(rho = 0.98)
```

For example, at $\theta = (1, 1)^\top$:

```r
theta_test <- c(1, 1)
target$log_pi(theta_test)
```

```
[1] -0.5050505
```

```
target$grad_log_pi(theta_test)
```

```
[1] -0.5050505 -0.5050505
```

Task 4

Transform to

$$u = \frac{\theta_1 + \theta_2}{\sqrt{2}}, \qquad v = \frac{\theta_1 - \theta_2}{\sqrt{2}}.$$

Calculate $\mathrm{Var}(u)$, $\mathrm{Var}(v)$, and $\mathrm{Cov}(u, v)$.

Answer:

In matrix form,

$$\begin{pmatrix} u \\ v \end{pmatrix} = A\theta, \qquad A = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

The transformed covariance matrix is

$$\mathrm{Var} \begin{pmatrix} u \\ v \end{pmatrix} = A\Sigma A^\top.$$

Using the general covariance formulas,

$$\mathrm{Var}(u) = \frac{\sigma_{11} + 2\sigma_{12} + \sigma_{22}}{2} = 1 + \rho,$$

$$\mathrm{Var}(v) = \frac{\sigma_{11} - 2\sigma_{12} + \sigma_{22}}{2} = 1 - \rho,$$

and

$$\mathrm{Cov}(u, v) = \frac{\sigma_{11} - \sigma_{22}}{2} = 0.$$

```
A <- matrix(c(1, 1, 1, -1), nrow = 2, byrow = TRUE) / sqrt(2)
Sigma_uv <- A %*% target$Sigma %*% t(A)
Sigma_uv
```

```
     [,1] [,2]
[1,] 1.98 0.00
[2,] 0.00 0.02
```

```r
transform_summary <- data.frame(
  Quantity = c("Var(u)", "Var(v)", "Cov(u,v)"),
  Value = c(Sigma_uv[1, 1], Sigma_uv[2, 2], Sigma_uv[1, 2])
)
transform_summary
```

Table 3 | Task 4: Variances and covariance in the transformed coordinates.

Quantity	Value
Var(u)	1.98
Var(v)	0.02
Cov(u,v)	0.00

Thus, for $\rho = 0.98$,

$$\mathrm{Var}(u) = 1.98, \qquad \mathrm{Var}(v) = 0.02, \qquad \mathrm{Cov}(u, v) = 0.$$

Task 5

Use your result to explain why reparameterization or a suitable HMC mass matrix can substantially improve computation.

Answer:

The transformation in Task 4 removes correlation but does not remove the scale difference. The standardized coordinates

$$z_u = \frac{u}{\sqrt{1+\rho}}, \qquad z_v = \frac{v}{\sqrt{1-\rho}}$$

have covariance matrix I. Equivalently,

$$z = \Sigma^{-1/2}\theta, \qquad z \sim N(0, I),$$

with inverse transformation

$$\theta = \Sigma^{1/2} z.$$

This is a whitening reparameterization: the contours become circles, so an isotropic HMC mass matrix is appropriate.

If HMC is run in the original θ coordinates, use the convention

$$K(p) = \frac{1}{2} p^\top M^{-1} p.$$

For this Gaussian target, the appropriate preconditioning is

$$M^{-1} = \Sigma, \qquad M = \Sigma^{-1}.$$

The following R code constructs these transformations and verifies that the whitened target has identity covariance.

```
# Define the target covariance matrix
rho <- 0.98

Sigma <- matrix(c(1, rho, rho, 1), nrow = 2)

# Precision matrix
precision <- solve(Sigma)

# Eigen decomposition
eig <- eigen(Sigma, symmetric = TRUE)

# Construct Sigma^(1/2) and Sigma^(-1/2)
Sigma_sqrt <- eig$vectors %*%
  diag(sqrt(eig$values)) %*%
  t(eig$vectors)

Sigma_inv_sqrt <- eig$vectors %*%
  diag(1 / sqrt(eig$values)) %*%
  t(eig$vectors)

# Whitening and inverse whitening transformations
whiten <- function(theta) {
  as.vector(Sigma_inv_sqrt %*% as.vector(theta))
}

unwhiten <- function(z) {
  as.vector(Sigma_sqrt %*% as.vector(z))
}

# HMC mass matrices under
# K(p) = 1/2 p^T M^{-1} p
M_inv <- Sigma
M <- precision

# Verify that Sigma_sqrt %*% Sigma_sqrt = Sigma
Sigma_sqrt %*% Sigma_sqrt
```

```
     [,1] [,2]
```

```
[1,] 1.00 0.98
[2,] 0.98 1.00
```

```
# Verify that Sigma_inv_sqrt %*% Sigma_inv_sqrt = Sigma^{-1}
Sigma_inv_sqrt %*% Sigma_inv_sqrt
```

```
          [,1]      [,2]
[1,]  25.25253 -24.74747
[2,] -24.74747  25.25253
```

```
# Verify that the whitened covariance is the identity
cov_whitened <- Sigma_inv_sqrt %*%
  Sigma %*%
  Sigma_inv_sqrt

cov_whitened
```

```
              [,1]          [,2]
[1,]  1.000000e+00 -8.881784e-16
[2,] -8.881784e-16  1.000000e+00
```

```
# Test the transformation with theta = (1,1)
theta_test <- c(1, 1)
z_test <- whiten(theta_test)
theta_back <- unwhiten(z_test)

z_test
```

```
[1] 0.7106691 0.7106691
```

```
theta_back
```

```
[1] 1 1
```

```
# Eigenvalue ratios
eigenvalue_ratio <- max(eig$values) / min(eig$values)
whitened_ratio <- max(eigen(cov_whitened)$values) /
  min(eigen(cov_whitened)$values)

eigenvalue_ratio
```

```
[1] 99
```

```
whitened_ratio
```

```
[1] 1
```

The original eigenvalue ratio is

$$\frac{\lambda_{\max}}{\lambda_{\min}} = \frac{1.98}{0.02} = 99,$$

whereas the whitened target has ratio 1. Reparameterization or a correctly chosen mass matrix therefore reduces the anisotropy, allows HMC trajectories to take longer and better-shaped steps, and can substantially reduce the number of required computations.

Exercise 3: Variational approximation and a missing dependence

Suppose the target distribution is

$$p(\theta) = N_2(0, \Sigma) \qquad \Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix},$$

with $|\rho| < 1$. We approximate it by a mean-field family

$$q(\theta_1, \theta_2) = q_1(\theta_1) q_2(\theta_2).$$

Task 1

Starting from the CAVI identity

$$\log q_j^*(\theta_j) = E_{q-j}\{\log p(\theta)\} + constant,$$

derive the optimal forms of q_1 and q_2.

Answer:

The target distribution is the centered bivariate normal

$$\theta \sim N_2(0, \Sigma), \qquad \Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix},$$

with precision matrix

$$\Sigma^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}.$$

Ignoring additive constants, the log-density is therefore

$$\log p(\theta) = -\frac{1}{2}\theta^\top \Sigma^{-1}\theta = -\frac{1}{2(1-\rho^2)}\left(\theta_1^2 + \theta_2^2 - 2\rho\theta_1\theta_2\right).$$

Under the mean-field approximation,

$$q(\theta) = q_1(\theta_1)q_2(\theta_2),$$

and the variational objective is the ELBO,

$$\mathrm{ELBO}(q) = \mathbb{E}_q[\log p(\theta)] - \mathbb{E}_q[\log q(\theta)].$$

To see why the CAVI identity gives the optimal factor, hold q_2 fixed and optimize only q_1. Define

$$h_1(\theta_1) = \exp\left(\mathbb{E}_{q_2}[\log p(\theta)]\right).$$

After normalizing h_1, the part of the ELBO depending on q_1 can be written as

$$\mathrm{ELBO}(q_1) = -\mathrm{KL}\left(q_1 \middle\| \frac{h_1}{Z_1}\right) + \text{constant}.$$

Because the KL divergence is nonnegative and equals zero only when its two arguments coincide, the ELBO is maximized by choosing

$$q_1^*(\theta_1) = \frac{h_1(\theta_1)}{Z_1} \propto \exp\left(\mathbb{E}_{q_2}[\log p(\theta)]\right).$$

Taking logarithms gives the CAVI update,

$$\log q_1^*(\theta_1) = \mathbb{E}_{q_2}[\log p(\theta)] + \text{constant}.$$

Thus, for a fixed q_2, no other choice of q_1 can produce a higher ELBO. The same argument holds symmetrically for q_2.

To obtain the explicit form of q_1, let

$$m_2 = \mathbb{E}_{q_2}(\theta_2), \qquad s_2^2 = \mathrm{Var}_{q_2}(\theta_2).$$

Then

$$\mathbb{E}_{q_2}(\theta_2) = m_2, \qquad \mathbb{E}_{q_2}(\theta_2^2) = m_2^2 + s_2^2.$$

Substitution into the log-density gives

$$\mathbb{E}_{q_2}[\log p(\theta)] = -\frac{1}{2(1-\rho^2)}\left[\theta_1^2 - 2\rho m_2\theta_1 + m_2^2 + s_2^2\right].$$

The term $m_2^2 + s_2^2$ does not depend on θ_1, so it is absorbed into the normalizing constant. Completing the square gives

$$\theta_1^2 - 2\rho m_2\theta_1 = (\theta_1 - \rho m_2)^2 - \rho^2 m_2^2,$$

and hence

$$\log q_1^*(\theta_1) = -\frac{(\theta_1 - \rho m_2)^2}{2(1-\rho^2)} + \text{constant}.$$

This is the log-density of a Gaussian with mean ρm_2 and variance $1-\rho^2$, so

$$q_1^*(\theta_1) = N\left(\rho m_2,\, 1-\rho^2\right).$$

Defining $m_1 = \mathbb{E}_{q_1}(\theta_1)$, the symmetric update for the second factor is

$$q_2^*(\theta_2) = N\left(\rho m_1,\, 1-\rho^2\right).$$

These factors are optimal within the restricted mean-field family, although they do not generally reproduce the true posterior because the product family cannot represent the correlation between the two coordinates. The variance $1-\rho^2$ is the conditional variance given the other coordinate, rather than the marginal variance; the mean-field approximation is exact only when $\rho = 0$.

Task 2

Show that at the CAVI fixed point both variational means are zero.

Answer:

From `Task 1`, the optimal variational distributions are

$$q_1^*(\theta_1) = N\left(\rho m_2,\, 1-\rho^2\right),$$

and

$$q_2^*(\theta_2) = N\left(\rho m_1,\, 1-\rho^2\right),$$

where

$$m_1 = \mathbb{E}_{q_1}(\theta_1), \qquad m_2 = \mathbb{E}_{q_2}(\theta_2).$$

At a CAVI fixed point, the means no longer change after an update. Therefore, they must satisfy

$$m_1 = \rho m_2, \qquad m_2 = \rho m_1.$$

Substitute the second equation into the first:

$$m_1 = \rho(\rho m_1) = \rho^2 m_1.$$

Rearranging gives

$$(1 - \rho^2) m_1 = 0.$$

Because the problem assumes

$$|\rho| < 1,$$

we know that

$$1 - \rho^2 > 0.$$

Thus, the only possible solution is

$$m_1 = 0.$$

Then, using $m_2 = \rho m_1$,

$$m_2 = 0.$$

Therefore, the fixed-point means are

$$\boxed{m_1 = 0, \qquad m_2 = 0.}$$

Task 3

Find the variational marginal variances.

Answer:

From `Task 1`, the optimal variation factors are

$$q_1^*(\theta_1) = N\left(\rho m_2,\, 1-\rho^2\right),$$

and

$$q_2^*(\theta_2) = N\left(\rho m_1,\, 1-\rho^2\right),$$

where

$$m_1 = \mathbb{E}_{q_1}(\theta_1), \qquad m_2 = \mathbb{E}_{q_2}(\theta_2).$$

Task 2 shows that, at the CAVI fixed point,

$$m_1 = 0, \qquad m_2 = 0.$$

Substituting these means into the variational distributions gives

$$q_1^*(\theta_1) = N(0,\, 1-\rho^2),$$

and

$$q_2^*(\theta_2) = N(0,\, 1-\rho^2).$$

For a Gaussian distribution $N(\mu, v)$, the second parameter v is its variance. Therefore,

$$\mathrm{Var}_{q^*}(\theta_1) = 1-\rho^2,$$

and

$$\mathrm{Var}_{q^*}(\theta_2) = 1-\rho^2.$$

The full variational approximation is thus

$$q^*(\theta) = N_2\left(0, \begin{pmatrix} 1-\rho^2 & 0 \\ 0 & 1-\rho^2 \end{pmatrix}\right).$$

Task 4

Compare these variances with the true marginal variances. What happens as $|\rho|$ approaches one?

Answer:

From Task 3, the variational marginal variances are

$$\mathrm{Var}_{q^*}(\theta_1) = 1 - \rho^2, \qquad \mathrm{Var}_{q^*}(\theta_2) = 1 - \rho^2.$$

Because

$$\rho^2 \geq 0,$$

we have

$$1 - \rho^2 \leq 1.$$

Equality holds only when

$$\rho = 0.$$

Thus, whenever the two variables are correlated, the mean-field approximation underestimates their marginal variances.

The reason is that $1 - \rho^2$ is a conditional variance. For example,

$$\mathrm{Var}(\theta_1 \mid \theta_2) = 1 - \rho^2.$$

This is the variance of θ_1 after θ_2 is known. The mean-field approximation incorrectly uses this conditional variance as if it were the marginal variance, thereby ignoring the uncertainty associated with θ_2 itself.

As

$|\rho| \to 1,$

the variational variance satisfies

$$1 - \rho^2 \to 0,$$

whereas the true marginal variance remains

1.

Therefore, under strong correlation, the mean-field approximation becomes increasingly overconfident and eventually collapses toward a point mass, even though each variable still has marginal variance one in the true target distribution.

The final comparison is

$$\boxed{\text{True marginal variances} = 1, \qquad \text{Variational variances} = 1 - \rho^2.}$$

The mean-field approximation is exact when $\rho = 0$, but its variance underestimation becomes increasingly severe as $|\rho|$ approaches one.

Task 5

Verify the result numerically in R for $\rho = 0, 0.5, 0.95$.

Answer:

```r
cavi_bvn <- function(rho,
                     n_iter = 200,
                     m1_init = 1,
                     m2_init = -1) {
  var_q <- 1 - rho^2
  m1 <- numeric(n_iter + 1)
  m2 <- numeric(n_iter + 1)
  m1[1] <- m1_init
  m2[1] <- m2_init

  for (b in seq_len(n_iter)) {
    # Update q1 using the current mean of q2
    m1[b + 1] <- rho * m2[b]

    # Update q2 using the updated mean of q1
    m2[b + 1] <- rho * m1[b + 1]
  }

  list(
    m1 = m1,
    m2 = m2,
    var_q = var_q
  )
}
```

```r
rho_values <- c(0, 0.5, 0.95)

cavi_results <- do.call(rbind, lapply(rho_values, function(rho) {
```

```
  fit <- cavi_bvn(rho = rho, n_iter = 200)
  data.frame(
    rho = rho,
    true_variance = 1,
    variational_variance = fit$var_q,
    final_m1 = tail(fit$m1, 1),
    final_m2 = tail(fit$m2, 1)
  )
}))
```

```
cavi_results_print <- cavi_results
cavi_results_print$final_m1 <- round(cavi_results_print$final_m1, 8)
cavi_results_print$final_m2 <- round(cavi_results_print$final_m2, 8)
cavi_results_print
```

Table 4 | Task 5: Numerical CAVI verification for rho = 0, 0.5, and 0.95.

rho	true_variance	variational_variance	final_m1	final_m2
0.00	1	1.0000	0	0
0.50	1	0.7500	0	0
0.95	1	0.0975	0	0

When $\rho = 0$, the two variables are already independent, so the mean-field approximation is exact and the variance is one. When $\rho = 0.5$, the variational variance is

$$1 - 0.5^2 = 0.75,$$

which is smaller than the true marginal variance. When $\rho = 0.95$, the variational variance is

$$1 - 0.95^2 = 0.0975,$$

which is much smaller than one, showing that the mean-field approximation becomes highly overconfident under strong correlation.

Thus, the numerical results verify that

$$\boxed{m_1 = m_2 = 0, \qquad \mathrm{Var}_{q^*}(\theta_1) = \mathrm{Var}_{q^*}(\theta_2) = 1 - \rho^2.}$$

Exercise 4: MCMC or variational inference?

Task 1

Explain why a stable ELBO does not establish that the VI approximation is close to the posterior.

Answer:

ELBO is the training score of VI. Its stability merely indicates that the optimization has converged. Giving similar Elbos from different starting points only indicates that the same stable solution has been repeatedly found.

However, convergence does not equal posterior accuracy. VI can only be selected within a limited family of approximations. For instance, mean-field assumes that the coefficients are independent, but the true posterior may be strongly correlated.

Just like fitting a curve with a straight line: repeatedly fitting leads to the same "optimal straight line", the process is very stable, but a straight line can never represent a curve.

$$\log p(y) = \mathrm{ELBO}(q) + \mathrm{KL}(q(\beta) \,\|\, p(\beta \mid y)).$$

When ELBO is stable, we usually do not know how large KL is, because true $logp(y)$ is unknown.

Therefore, a stable ELBO only indicates convergence within the selected family and does not mean it is close to the true posterior. VI may find the best mean-field approximation, but this approximation is still too simple and even misleading.

Task 2

Give a plausible explanation for the smaller VI standard deviation in the table.

Answer:

The mean-field VI assumes that all coefficients are independent of each other. That is to say, in the approximate posterior, when one coefficient increases or decreases, it will not change along with the other coefficients. However, in Bayesian logistic regression, there is often a posterior correlation among coefficients, especially when the predictor variables themselves are correlated or when several predictor variables provide overlapping information.

For instance, if there are two highly correlated predictor variables, the data might effectively constrain their "combined effect", but it is impossible to clearly distinguish which variable is at play. The true posterior will allow one coefficient to be larger and the other smaller, so the joint posterior will be elongated, and the edge uncertainty of a single coefficient will also be relatively large. The mean-field VI cannot represent this kind of relationship that changes together, so this part of uncertainty is discarded, making the distribution too narrow.

This corresponds to the results in the table: the standard deviation of NUTS is 0.22, 0.22, while the standard deviation of VI is only 0.13, 0.13. The mean values of the two are very close, indicating that VI's estimation of the coefficient position is acceptable, but it underestimates the uncertainty. Therefore, $P(\beta > 0) = 0.99$ given by VI is more extreme than the 0.92 given by NUTS, appearing overly confident.

Simply, the standard deviation of VI is smaller because the mean-field assumption ignores the posterior correlation between coefficients, thereby compressing the joint uncertainty in the true posterior.

Task 3

Both coefficients come from the same variational fit, yet the approximation is accurate for one and not the other. Give a plausible explanation.

Answer:

The posterior shapes of different coefficients are different, so the assumption of the independence of mean-field has different influences on them. If the predictor variable corresponding to a certain coefficient is independent, well-informed, and almost uncorrelated with other variables, and its posterior is close to independent Gaussian, VI can accurately estimate the mean and SD. If this coefficient is highly correlated with or approximately collinear with other variables, the data can only constrain the common effect, and the coefficients will waxing and waning. mean-field does not allow such joint changes, and thus obviously underestimates SD. Therefore, a close mean does not imply that SD and the tail probability are accurate, and the accuracy of one coefficient does not mean that the entire VI is approximately accurate. The second question is that the mean is close, but SD is significantly underestimated.

Task 4

If the scientific goal is accurate posterior uncertainty for a small number of coefficients, which additional checks would you perform before using the VI results?

Answer:

Before making scientific inferences with VI, first use NUTS as a reference on representative models or data. It is not only necessary to compare the mean, but also the SD, median, confidence interval, quantile, tail probability (such as $P(\beta > 0)$) and coefficient sign, because VI may be accurate in the mean but underestimate the uncertainty.

Let's take a look at the posterior correlation or covariance matrix of NUTS. If the key coefficients are strongly correlated, the independent assumption of mean-field is unreliable, and SD is prone to be too small. At this time, full-rank Gaussian VI or MCMC can be considered.

The initial values should also be changed and re-fitted. Posterior prediction checks, reserved prediction, prior and model sensitivity analyses should be conducted, and the interval coverage rate should be checked with simulated data. First, determine the acceptable errors, such as SD, interval width, and the maximum difference in tail probability. VI should only be used

when the standards are met in representative cases. The model ultimately used for scientific conclusions still needs to be confirmed by MCMC.

Task 5

If the goal is to fit thousands of similar models for exploratory screening, explain why VI might nevertheless be useful.

Answer:

VI is still useful in large-scale exploratory screening because the primary goal at this time is to compare quickly and consistently, rather than having the uncertainty of complete calibration for each coefficient. For instance, each NUTS model takes 25 minutes, while VI only takes 20 seconds, which is approximately 75 times faster. Thousands of models can condense weeks of calculation to just a few hours.

During the screening stage, VI can provide the approximate effect size, prediction probability, model ranking, as well as notable coefficients or model labels. Even if the SD is relatively narrow, it is still suitable for triage to pick out candidates that need to be examined in detail.

The reasonable process is: VI conducts large-scale screening, and then uses NUTS to review a few selected or scientifically important models. This way, not only is the computational advantage of VI retained, but the final conclusion is also based on more reliable posterior uncertainties.

Task 6

Write a short paragraph explaining why the question "Is MCMC or VI better?" has no universal answer.

Answer:

Which is better, MCMC or VI? There is no standard answer. It depends on what your goal and the background look like, as well as whether your computing power is sufficient. MCMC can provide a complete posterior asymptotically valid sample and also characterize correlations and non-Gaussian shapes, but it may be very expensive. VI is usually much faster and easier to run on a large number of models, but it only looks within limited approximations and may underestimate uncertainty. So, if the ultimate inference is about calibrated uncertainty, MCMC is generally more reliable. If it's just for quick exploration or large-scale screening, VI is often more suitable. When actually doing it, it is best to combine the two: first, use VI to identify promising models, and then use MCMC to review the final results.

Exercise 5: Norppa course feedback

